Отчёт по лабораторной работе № 4

Профессиональная работа с Git: Gitflow и релизы

Abstract

В данном отчёте представлены результаты выполнения лабораторной работы по теме: освоение рабочего процесса Gitflow, семантического версионирования и Conventional Commits.

Описаны установка и настройка инструментов, создание тестового репозитория, реализация двух релизов с автоматической генерацией CHANGELOG.md и публикация их на GitHub.

Сделаны выводы о достижении поставленной цели и о применении полученных навыков в профессиональной командной разработке.

Введение

1. Цель работы
2. Теоретические сведения > 2.1. Рабочий процесс Gitflow > 2.1.1. Общая информация > 2.1.2. Процесс работы con Gitflow > 2.2. Семантическое версионирование > 2.2.1. Краткое описание семантического версионирования > 2.2.2.Программное обеспечение > 2.3. Общепринятые коммиты > 2.3.1. Описание > 2.3.2. Типы коммитов
3. Задание и последовательность выполнения работы 3.1. Установка программного обеспечения > 3.1.1. Установка git-flow > 3.1.2. Установка Node.js > 3.1.3. Настройка Node.js > 3.2. Общепринятые коммиты > 3.2.1. Установка commitizen > 3.2.2. Установка standard-changelog > 3.3. Практический сценарий использования git > 3.3.1. Создание репозитория git > 3.3.2. Работа репозитория git
4. Выводы
5. Список литературы

Цель работы

Целью данной лабораторной работы является получение навыков правильной работы с репозиториями git, освоение рабочего процесса Gitflow, изучение

семантического версионирования и правил оформления коммитов в соответствии со спецификацией Conventional Commits. В ходе работы необходимо установить и настроить программное обеспечение для работы с Gitflow, изучить структуру ветвления, освоить создание функциональных веток, веток релиза и исправлений, а также научиться автоматизировать процесс ведения журнала изменений с помощью стандартных инструментов.

Теоретические сведения

В этом разделе представлены основные концепции Gitflow, семантического управления версиями и соглашений по оформлению сообщений коммитов, необходимые для понимания принципов разработки.

2.1 Рабочий процесс Gitflow

2.1.1 Общая информация

Gitflow — это модель ветвления для Git, предназначенная для организации процесса разработки ПО вокруг запланированных релизов. Разработанная Винсентом Дриссеном в 2010 году, она стала отраслевым стандартом для проектов, требующих структурированного цикла публикаций. Эта модель определяет стратегию ветвления, которая позволяет упорядочено управлять разработкой новых функций, подготовкой версий и срочными исправлениями

Структура Gitflow основана на двух основных ветвях неопределённой продолжительности:

- Основной или основной: содержит готовый к производству код, стабильную версию проекта. Все официальные релизы помечены в этой ветке.
- develop-служит ветвью интеграции для новых функций. В нем объединяются изменения функциональных ветвей, и он является основой для подготовки релизов.

В дополнение к этим основным ветвям Gitflow использует несколько типов вспомогательных ветвей, которые являются временными :

- Функциональные ветки (feature/*): создаются на основе ветки develop для разработки новых функций. После завершения работы они снова объединяются с веткой develop.

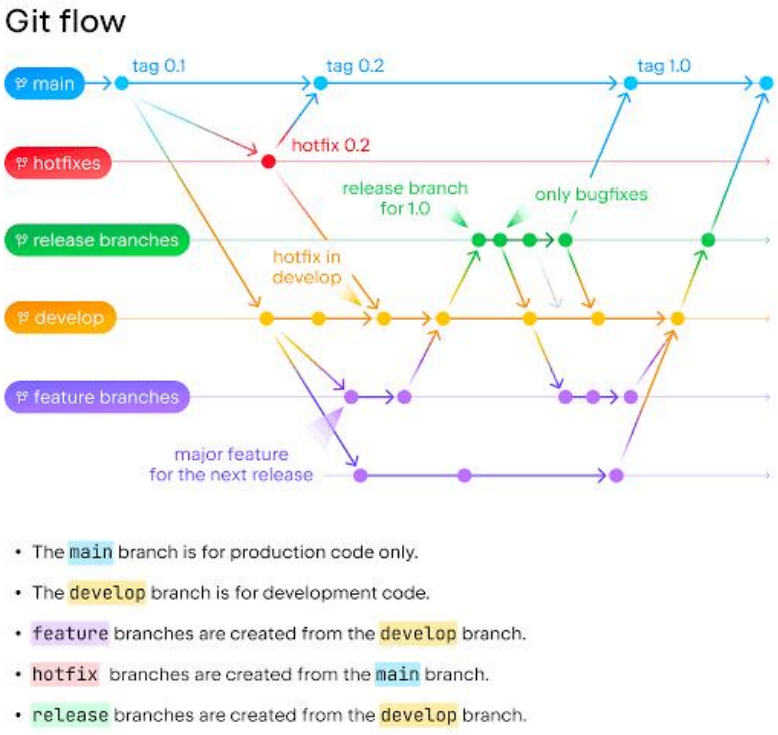


Figure 1: Gitflowimage

- Веточки релизов (release/*): формируются на основе ветки develop, когда в системе накапливается достаточно функционала для нового релиза. На этом этапе допускаются только исправления ошибок и подготовка метаданных. По завершении они объединяются в основную ветку (с соответствующей меткой версии) и в ветку разработки.
- Ветви срочных исправлений (hotfix/*): создаются на основе основной ветки для устранения критических ошибок в производственной среде. После завершения работы они также объединяются в основную ветку и в ветку разработки.

2.1.2 Процесс работы con Gitflow

Рабочий цикл с Gitflow можно резюмировать в следующих шагах:

1. Инициализация: структура настраивается путём запуска git flow init. Это устанавливает ветви main и develop и определяет префиксы для вспомогательных ветвей (feature/, release/, hotfix/).
2. Разработка функциональности: новая ветка функциональности запускается с помощью git flow feature start имя-функциональность. Над ней ведётся работа, и по завершении она интегрируется в ветку develop с помощью команды git flow feature finish имя-функциональности.
3. Подготовка к релизу: при планировании выпуска новой версии создаётся специальная ветка релиза с помощью команды git flow release start 1.0.0. В этой ветке проводятся финальные настройки и обновляется документация. По завершении выполняется команда git flow release finish 1.0.0, которая объединяет изменения в ветках main и develop и создаёт тег с номером версии.
4. Устранение критических ошибок: при обнаружении серьёзной проблемы в продакшене создаётся ветка hotfix на основе main с помощью команды git flow hotfix start fix-urgent. После внедрения исправления выполняется финальная команда git flow hotfix finish fix-urgent, которая объединяет эту ветку с main и develop.

Этот рабочий процесс обеспечивает чёткую структуру, упрощающую командную работу и управление версиями.

2.2. Семантическое версионирование

2.2.1. Краткое описание семантического версионирования

Семантическое управление версиями (SemVer) - это набор правил, которые определяют, как номера версий назначаются и увеличиваются в программном

обеспечении, с целью информирования о типе внесённых изменений . В их спецификации указано, что версия должна быть отформатирована как САМАЯ БОЛЬШАЯ.МЛАДШИЙ.ПАТЧ (X. Y. Z), где:

- МАЙОР (X): увеличивается при внесении несовместимых с предыдущими версиями изменений в публичный API.
- МИНОР (Y): увеличивается при добавлении новой функциональности, совместимой с предыдущими версиями.
- ПАЧЧЕ (Z): увеличивается при исправлении ошибок, сохраняющих совместимость с предыдущими версиями.

Эта конвенция позволяет разработчикам и инструментам управления зависимостями оценивать влияние новой версии без необходимости детального анализа кода. Благодаря использованию SemVer можно избежать «кошмара зависимостей», задавая диапазоны версий, которые гарантированно совместимы.

2.2.2. Программное обеспечение

Существует множество инструментов для внедрения семантического управления версиями. Один из самых популярных — это standard-version или пакет standard-changelog, которые автоматизируют создание файла CHANGELOG.md на основе сообщений коммитов и помогают соблюдать принцип SemVer при увеличении версий. Другие инструменты, такие как semantic-release, также автоматизируют процесс публикации, опираясь на принципы Conventional Commits.

2.3. Общепринятые коммиты

2.3.1. Описание

Спецификация Conventional Commits - это упрощённое соглашение для сообщений о фиксации. Он предоставляет набор правил для создания явной истории коммитов, что упрощает написание автоматизированных инструментов. Это соглашение тесно связано с SemVer, поскольку оно описывает в сообщении фиксации тип внесённого изменения (новая функциональность, исправление ошибок или несовместимое изменение).

Формат обычного сообщения о фиксации следующий:

```
< >(<           >): <    >
[             ]
[             ]
```

Основными элементами являются :

- исправить:: фиксация типа fix исправляет ошибку в коде. Это соответствует приращению патча в SemVer.
- feat:: фиксация типа feat добавляет новую функциональность. Это соответствует МЕНЬШЕМУ приращению в Севере.
- BREAKING CHANGE:: фиксация, содержащая текст BREAKING CHANGE: в начале или нижней части сообщения, указывает на изменение, несовместимое с предыдущими версиями. Это соответствует большему приращению в Севере.

Помимо этих основных типов, для классификации других видов изменений рекомендуется использовать такие категории, как docs:, style:, refactor:, perf: и test:.

2.3.2. Типы коммитов

Основными типами коммитов в соответствии с обычной спецификацией являются:

- Тип Описание Корреляция SemVer
- feat Добавляет в код новую функциональность. НЕЗНАЧИТЕЛЬНЫЙ
- исправление Исправляет ошибку в коде. ЗАПЛАТКА
- BREAKING CHANGE Вносит изменение, нарушающее совместимость API. Он может быть частью любого типа коммита. МАЙОР

В дополнение к основным типам для поддержания чистой и семантической историикоммитов обычно используются следующие типы:

- документы:: Изменения в документации.
- стиль:: Изменения, которые не влияют на смысл кода (форматирование, пробелы и т. Д.).
- рефакторинг:: Изменения в коде, которые не исправляют ошибку и не добавляют функциональности.
- perf:: Улучшения производительности.
- тест:: Добавление или исправление тестов.
- работа по дому: Изменения в процессе строительства или вспомогательные инструменты.

Задание и последовательность выполнения работы

В этом разделе описаны практические шаги, необходимые для выполнения лабораторной работы — от установки инструментов до создания релизов и публикации на GitHub.

3.1. Установка программного обеспечения

3.1.1. Установка git-flow

Git-flow-это набор расширений для Git, который упрощает реализацию модели ветвления Gitflow. Чтобы установить его в Fedora, сначала необходимо включить репозиторий CopR, содержащий пакет:

```
alexei@fedora:~$ sudo dnf copr enable elegos/gitflow
[sudo] contraseña para alexei:
 https://copr.fedorainfracloud.org/api_3/rpmr 100% | 478.0   B/s | 560.0   B |  00m01s
Va a habilitar un repositorio Copr. Tenga en cuenta que no es parte de
la distribución, y que no hay garantías de calidad.

El Proyecto Fedora no ejerce ningún control sobre el contenido de este
repositorio, más allá de las reglas publicadas en el P+F de Copr en
<https://docs.pagure.org/copr.copr/user_documentation.html#what-i-can-build-in-copr>,
y no se comprueba la calidad ni la seguridad de los paquetes.

No envíe informes de errores sobre estos paquetes al Bugzilla de Fedora.
Si observa problemas, contacte directamente con el propietario de este repositorio.
Is this ok [y/N]: y
alexei@fedora:~$ sudo dnf install gitflow -y
Actualizando y cargando repositorios:
 Fedora 44 - x86_64 - Updates                100% |   5.7 KiB/s |  17.2 KiB |  00m03s
 RPM Fusion for Fedora 44 - Nonfree - Steam  100% |   5.9 KiB/s |   8.4 KiB |  00m01s
 RPM Fusion for Fedora 44 - Nonfree - NVIDIA 100% |   7.6 KiB/s |   8.8 KiB |  00m01s
 google-chrome                               100% |   3.4 KiB/s |   1.3 KiB |  00m00s
 Fedora 44 - x86_64                          100% |  21.3 KiB/s |  20.4 KiB |  00m01s
 Copr repo for PyCharm owned by phracek      100% |   5.5 KiB/s |   2.1 KiB |  00m00s
 Copr repo for gitflow owned by elegos       100% |   1.8 KiB/s |   2.4 KiB |  00m01s
Repositorios cargados.
Paquete                 Arq.   Versión                  Repositorio            Tamaño
Instalando:
 gitflow                x86_64 0:1.12.3-1.fc34          copr:copr.fedorainf 261.7 KiB

Resumen de la transacción:
 Instalando:          1 paquete

El tamaño total de paquetes entrantes es 57 KiB. Se necesita descargar 57 KiB.
Después de esta operación, 262 KiB extra serán utilizados (instalar 262 KiB, eliminar
0 B).
[1/1] gitflow-0:1.12.3-1.fc34.x86_64         100% |  42.7 KiB/s |  57.4 KiB |  00m01s
----------------------------------------------------------------------------------------
[1/1] Total                                  100% |  39.9 KiB/s |  57.4 KiB |  00m01s
[1/2] https://download.copr.fedorainfracloud. ???% |   0.0   B/s |   0.0   B |  00m00s
[1/2] https://download.copr.fedorainfracloud. 100% |   9.2 KiB/s | 998.0   B |  00m00s
```

Проверка установки:

Чтобы убедиться в корректной установке git-flow, выполните команду:

```
alexei@fedora:~$ git flow version
1.12.3 (AVH Edition)
```

Figure 2: gitflow version image

3.1.2. Установка Node.js

Node.js это среда выполнения JavaScript, используемая для запуска инструментов обычных коммитов и генерации журналов изменений. В Fedora он устанавливается вместе с менеджером пакетов pnpm:

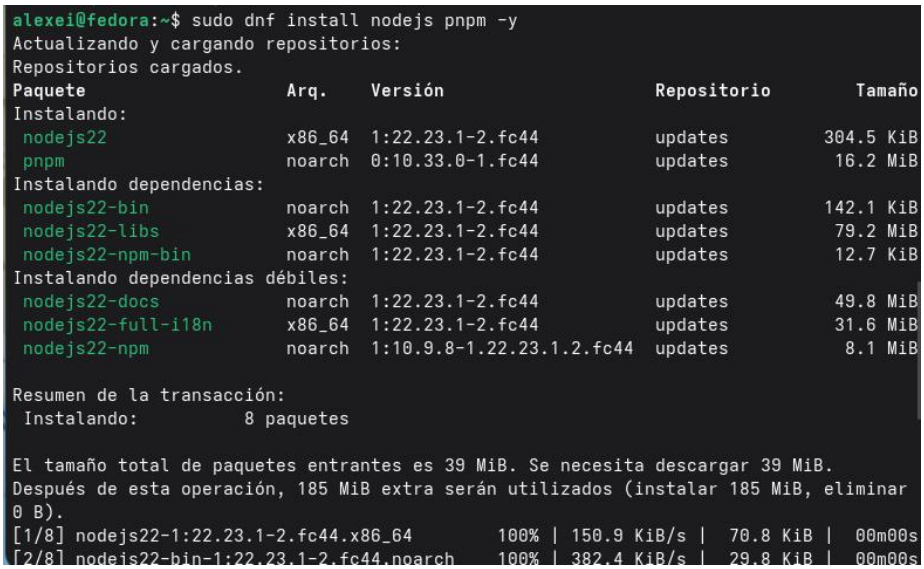


Figure 3: Node.js image

Проверка установки:

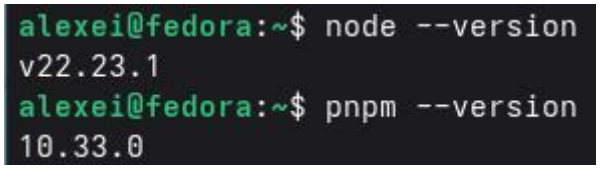


Figure 4: Node.js version image

3.1.3. Настройка Node.js

Чтобы глобально установленные пакеты с помощью pnpm были доступны в терминале, необходимо настроить переменную окружения PATH:

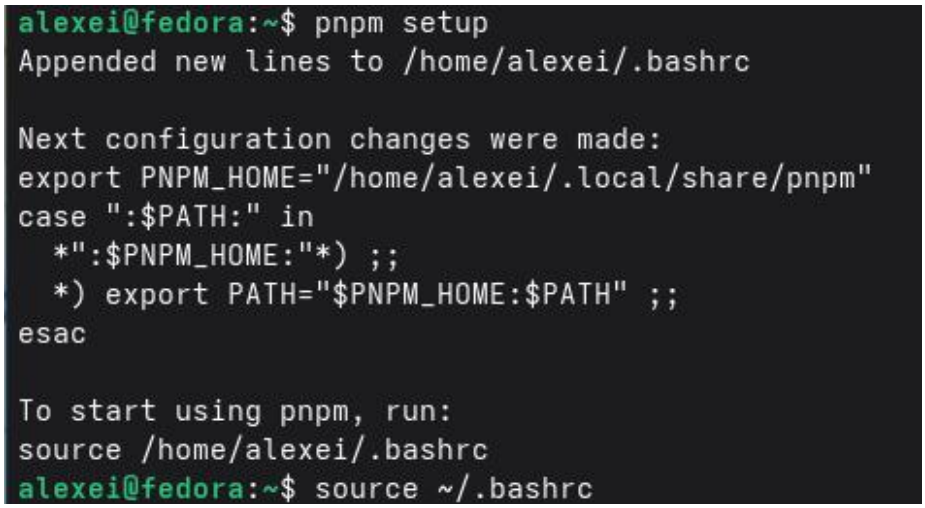


Figure 5: pnpm image

3.2. Общепринятые коммиты

3.2.1. Установка commitizen

Commitizen - это инструмент, который помогает создавать коммиты в формате обычных коммитов в интерактивном режиме. Устанавливается глобально с помощью pnpm:

3.2.2. Установка standard-changelog

Standard-changelog genera automáticamente un archivo CHANGELOG.md basado en los mensajes de commit. Se instala globalmente:

3.3. Практический сценарий использования git

3.3.1. Создание репозитория git

Шаг 1: Создание репозитория на GitHub

Шаг 2: Инициализация файла package.json

Шаг 3: Настройка файла package.json

Отредактируйте файл package.json, чтобы он содержал следующее содержимое:

Шаг 4. Выполните первую фиксацию с помощью commitizen

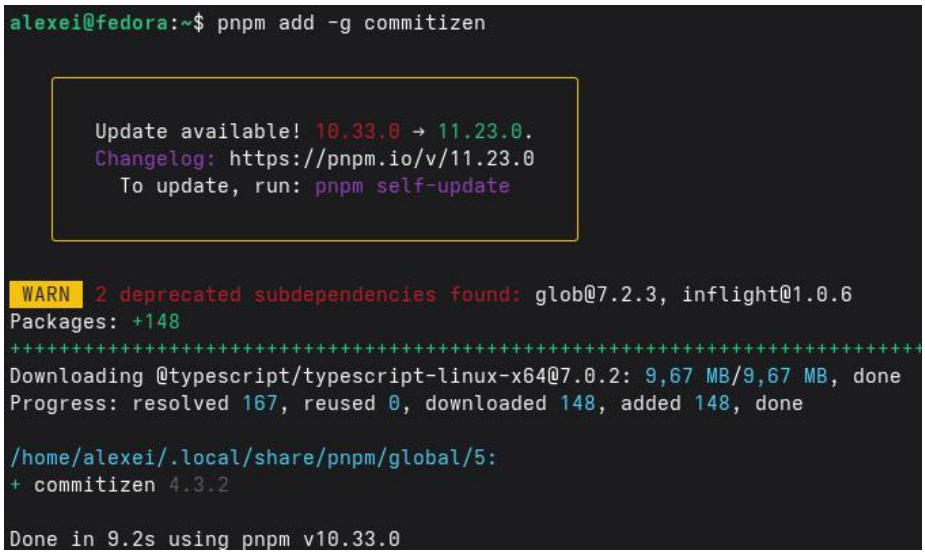


Figure 6: commitizen image

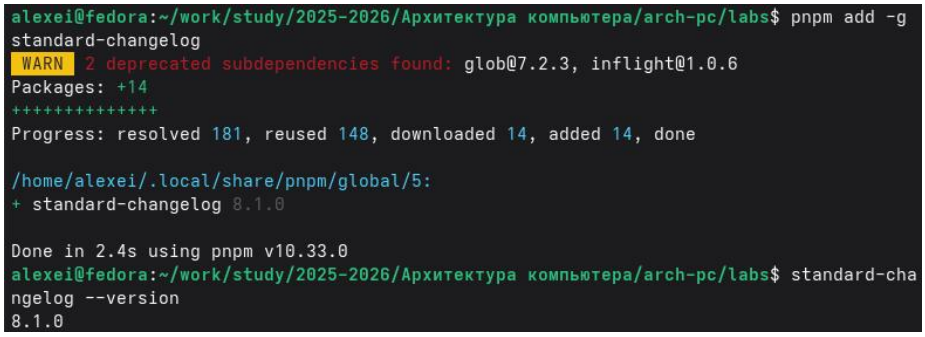


Figure 7: Standard-changelog image

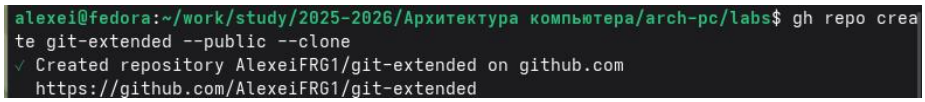


Figure 8: Шаг 1 image

```
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs$ cd git-exten
ded
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 pnpm init
Wrote to /home/alexei/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-ext
ended/package.json

{
  "name": "git-extended",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "packageManager": "pnpm@10.33.0"
}
```

Figure 9: IIIar 2 image

```
 GNU nano 8.7.1                    package.json
{
  "name": "git-extended",
  "version": "1.0.0",
  "description": "Repository Git for education",
  "main": "index.js",
  "repository": "git@github.com:AlexeiFRG1/git-extended.git",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "Fernando Alexei Romero Gaspar <1132255520@rudn.ru>",
  "license": "CC-BY-4.0",
  "packageManager": "pnpm@10.33.0",
  "config": {
        "commitizen":{
                "path": "cz-conventional-changelog"
                }
        }
}
```

Figure 10: IIIar 3 image

```
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git add .
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git cz
cz-cli@4.3.2, cz-conventional-changelog@3.3.0

? Select the type of change that you're committing: chore:    Other changes that don't
 modify src or test files
? What is the scope of this change (e.g. component or file name): (press enter to
skip) initial
? Write a short, imperative tense description of the change (max 84 chars):
 (37) starting configuration of the proyect
? Provide a longer description of the change: (press enter to skip)

? Are there any breaking changes? No
? Does this change affect any open issues? No
[false (commit-raíz) ce0a7ff] chore(initial): starting configuration of the proyect
 1 file changed, 19 insertions(+)
 create mode 100644 package.json
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git push -u origin main
Enumerando objetos: 3, listo.
Contando objetos: 100% (3/3), listo.
Compresión delta usando hasta 2 hilos
Comprimiendo objetos: 100% (2/2), listo.
Escribiendo objetos: 100% (3/3), 1.19 KiB | 1.19 MiB/s, listo.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:AlexeiFRG1/git-extended.git
 * [new branch]      main -> main
rama 'main' configurada para rastrear 'origin/main'.
```

3.3.2. Работа репозитория git

Шаг 1. Инициализируйте Gitflow

```
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git flow init

Which branch should be used for bringing forth production releases?
   - main
Branch name for production releases: [main]
Branch name for "next release" development: [develop] develop

How to name your supporting branch prefixes?
Feature branches? [feature/]
Bugfix branches? [bugfix/]
Release branches? [release/]
Hotfix branches? [hotfix/]
Support branches? [support/]
Version tag prefix? [] v
Hooks and filters directory? [/home/alexei/work/study/2025-2026/Архитектура компьютера
/arch-pc/labs/git-extended/.git/hooks]
```

Figure 11: Шаг 1-1 image

Загрузить все ветки в удаленный репозиторий:

Установить отслеживание ветки разработки:

```
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git push --all
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:      https://github.com/AlexeiFRG1/git-extended/pull/new/develop
remote:
To github.com:AlexeiFRG1/git-extended.git
 * [new branch]      develop -> develop
```

Figure 12: Шаг 1-2 image

```
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git branch --set-upstream-to=origin/develop develop
rama 'develop' configurada para rastrear 'origin/develop'.
```

Figure 13: Шаг 1-3 image

Шаг 2. Создайте первый релиз (версия 1.0.0)

```
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git flow release start 1.0.0
Cambiado a nueva rama 'release/1.0.0'

Summary of actions:
- A new branch 'release/1.0.0' was created, based on 'develop'
- You are now on branch 'release/1.0.0'

Follow-up actions:
- Bump the version number now!
- Start committing last-minute fixes in preparing your release
- When done, run:

     git flow release finish '1.0.0'

alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 standard-changelog --first-release
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git add CHANGELOG.md
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git commit -m "docs(changelog): add register of the starting changes"
[release/1.0.0 b1a73b7] docs(changelog): add register of the starting changes
 1 file changed, 1 insertion(+)
 create mode 100644 CHANGELOG.md
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git flow release finish 1.0.0
Cambiado a rama 'main'
Tu rama está actualizada con 'origin/main'.
Merge made by the 'ort' strategy.
 CHANGELOG.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 CHANGELOG.md
Ya en 'main'
Tu rama está adelantada a 'origin/main' por 2 commits.
  (usa "git push" para publicar tus commits locales)
fatal: ¿sin mensaje de tag?
Fatal: Tagging failed. Please run finish again to retry.
```

```
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git push --all
Enumerando objetos: 5, listo.
Contando objetos: 100% (5/5), listo.
Compresión delta usando hasta 2 hilos
Comprimiendo objetos: 100% (3/3), listo.
Escribiendo objetos: 100% (4/4), 1.81 KiB | 617.00 KiB/s, listo.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:AlexeiFRG1/git-extended.git
   ce0a7ff..d3ed52e  main -> main
 * [new branch]      release/1.0.0 -> release/1.0.0
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git push --tags
Everything up-to-date
```

```
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 gh release create v1.0.0 -F CHANGELOG.md
https://github.com/AlexeiFRG1/git-extended/releases/tag/v1.0.0
```

Шаг 3: Разработка новой функциональности

```
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git flow feature start feature_branch
Cambiado a nueva rama 'feature/feature_branch'

Summary of actions:
- A new branch 'feature/feature_branch' was created, based on 'develop'
- You are now on branch 'feature/feature_branch'

Now, start committing on your feature. When done, use:

     git flow feature finish feature_branch
```

```
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 echo "New funcionality add" > feature.txt
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git add feature.txt
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git cz
cz-cli@4.3.2, cz-conventional-changelog@3.3.0

? Select the type of change that you're committing: feat:     A new feature
? What is the scope of this change (e.g. component or file name): (press enter to
skip) core
? Write a short, imperative tense description of the change (max 88 chars):
 (25) "add new characteristics"
? Provide a longer description of the change: (press enter to skip)

? Are there any breaking changes? No
? Does this change affect any open issues? No
[feature/feature_branch f5ae017] feat(core): "add new characteristics"
 1 file changed, 1 insertion(+)
 create mode 100644 feature.txt
```

```
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git flow feature finish feature_branch
Cambiado a rama 'develop'
Tu rama está actualizada con 'origin/develop'.
Actualizando ce0a7ff..f5ae017
Fast-forward
 feature.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 feature.txt
Eliminada la rama feature/feature_branch (era f5ae017).

Summary of actions:
- The feature branch 'feature/feature_branch' was merged into 'develop'
- Feature branch 'feature/feature_branch' has been locally deleted
- You are now on branch 'develop'
```

Шаг 4: Создайте второй релиз (версия 1.2.3)

```
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git flow release start 1.2.3
Branches 'develop' and 'origin/develop' have diverged.
And local branch 'develop' is ahead of 'origin/develop'.
Cambiado a nueva rama 'release/1.2.3'

Summary of actions:
- A new branch 'release/1.2.3' was created, based on 'develop'
- You are now on branch 'release/1.2.3'

Follow-up actions:
- Bump the version number now!
- Start committing last-minute fixes in preparing your release
- When done, run:

    git flow release finish '1.2.3'
```

```
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 nano package.json
```

```
{
  "name": "git-extended",
  "version": "1.2.3",
  "description": "Repository Git for education",
  "main": "index.js",
  "repository": "git@github.com:AlexeiFRG1/git-extended.git",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "Fernando Alexei Romero Gaspar <1132255520@rudn.ru>",
  "license": "CC-BY-4.0",
  "packageManager": "pnpm@10.33.0",
  "config": {
        "commitizen":{
                "path": "cz-conventional-changelog"
                }
        }
}
```

```
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 standard-changelog
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git add CHANGELOG.md package.json
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git commit -m "chore(release):actualization of the version 1.2.3"
[develop ccf3989] chore(release):actualization of the version 1.2.3
 2 files changed, 2 insertions(+), 1 deletion(-)
```

```
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git flow release finish 1.2.3
Branches 'develop' and 'origin/develop' have diverged.
And local branch 'develop' is ahead of 'origin/develop'.
Cambiado a rama 'main'
Tu rama está actualizada con 'origin/main'.
Merge made by the 'ort' strategy.
 CHANGELOG.md | 6 ++++++
 feature.txt  | 1 +
 package.json | 2 +-
 3 files changed, 8 insertions(+), 1 deletion(-)
 create mode 100644 feature.txt
Ya en 'main'
Tu rama está adelantada a 'origin/main' por 5 commits.
  (usa "git push" para publicar tus commits locales)
Cambiado a rama 'develop'
Tu rama está adelantada a 'origin/develop' por 1 commit.
  (usa "git push" para publicar tus commits locales)
Merge made by the 'ort' strategy.
 CHANGELOG.md | 5 +++++
 1 file changed, 5 insertions(+)
Eliminada la rama release/1.2.3 (era 4a7c7a3).

Summary of actions:
```

```
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git push --all
Enumerando objetos: 15, listo.
Contando objetos: 100% (13/13), listo.
Compresión delta usando hasta 2 hilos
Comprimiendo objetos: 100% (8/8), listo.
Escribiendo objetos: 100% (9/9), 3.87 KiB | 3.87 MiB/s, listo.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:AlexeiFRG1/git-extended.git
   5af3bb5..21a489f  develop -> develop
   d3ed52e..8127c93  main -> main
```

```
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 git push origin v1.2.3 --force
Enumerando objetos: 1, listo.
Contando objetos: 100% (1/1), listo.
Escribiendo objetos: 100% (1/1), 171 byte | 171.00 KiB/s, listo.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:AlexeiFRG1/git-extended.git
 + d3ed52e...4f6bc17 v1.2.3 -> v1.2.3 (forced update)
```

```
alexei@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/git-extended$
 gh release edit v1.2.3 -F CHANGELOG.md
https://github.com/AlexeiFRG1/git-extended/releases/tag/v1.2.3
```

Выводы

В ходе выполнения лабораторной работы я освоил продвинутые методы работы с Git: изучил модель ветвления Gitflow, научился использовать семантическое версионирование и применять спецификацию Conventional Commits. Я установил и настроил необходимые инструменты (git-flow, Node.js, commitizen, standard-changelog), создал тестовый репозиторий, реализовал два релиза с автоматической генерацией changelog-файлов и опубликовал их на GitHub. Все поставленные задачи выполнены, полученные навыки будут полезны для профессиональной разработки в команде.

Список литературы

- Дриссен, В. (2010). Успешная модель ветвления в Git. nvie.com
- GitHub. (2026). CJ-Systems/gitflow-cjs: CJS-версия расширений для Git. GitHub
- Традиционные коммиты. (2020). Традиционные коммиты 1.0.0-beta.3
- Семантическое управление версиями. (2013). Семантическое управление версиями 2.0.0